ЛР 2 ABC — конспект для понимания

Oct 29, 2026 · @Sokrat

1. Общая картина: что мы вообще строили

В одной фразе: мы собрали мини-систему из двух компьютеров, где первый готовит данные, второй обучает на них модель, а ответ возвращается обратно первому по сети.

Это упрощённая модель того, как устроены настоящие распределённые системы: разные задачи живут на разных машинах, каждая в своём изолированном «ящике» (контейнере), а общаются они по сети.

Роли:

Узел	Кто	Адрес	Контейнер	Что делает
Ноутбук А	ты (`aven`)	10.210.125.97	`data-processor`	чистит CSV и отправляет его на Б, потом спрашивает у Б прогноз
Ноутбук Б	друг (`ilu-shaa`)	10.210.125.250	`ml-trainer`	принимает CSV, обучает модель, отвечает на HTTP-запросы

Пайплайн по шагам:

1. На А в контейнере появляется «сырой» файл `raw_data.csv` (у вас — набор Iris, 150 цветков ириса).
2. Скрипт на А чистит его и сохраняет `clean_data.csv`.
3. А передаёт файл на Б по **SSH/SCP** — зашифрованному каналу.
4. На Б контейнер видит этот файл, обучает **дерево решений** и поднимает маленький веб-сервер **Flask** на порту 5000.
5. А делает запрос `curl http://10.210.125.250:5000/predict` и получает ответ в формате JSON: прогнозы и точность 0,9333.
6. **iptables** на Б следит, чтобы к порту 5000 мог стучаться только А.

Ваши ноутбуки — это виртуальные машины с Ubuntu: у тебя UTM на Mac (ARM), у друга VirtualBox (x86). Для лабораторной это неважно: Linux он и есть Linux.

2. Сеть: как ноутбуки находят друг друга

Чтобы два компьютера общались, каждому нужен **IP-адрес** — как номер дома в сети. Первое, что мы делали, — узнавали адреса и проверяли, что ноутбуки видят друг друга.

`ip a` — показать сетевые интерфейсы

Интерфейс — это «сетевая карта», настоящая или виртуальная. В выводе у тебя было четыре штуки:

Интерфейс	Адрес	Что это
`lo`	127.0.0.1	«петля», компьютер сам с собой; наружу не виден
`enp0s1`	10.210.125.97/24	главный: через него ты в Wi-Fi-сети. Этот адрес и нужен другу
`docker0`	172.17.0.1/16	виртуальный мост Docker по умолчанию. `DOWN` = на нём сейчас нет контейнеров
`br-3d3d7a4f88c8`	172.18.0.1/16	мост, который создал Docker Compose для вашего контейнера (сеть `node_a_default`)

Как читать адрес `10.210.125.97/24`: `/24` — это маска, её первые 24 бита (`10.210.125`) — «номер сети», а последнее число — номер устройства. У друга `10.210.125.250/24`: сеть та же, значит, вы в одной локалке и можете говорить напрямую.

`ip -br -4 addr` — то же самое, но кратко (`-br` = brief) и только IPv4 (`-4`).

Почему у виртуалки адрес из Wi-Fi-сети

Виртуальная машина может быть подключена двумя способами:

- **NAT** (по умолчанию): виртуалка спрятана за ноутбуком, и другие устройства её не видят. У тебя в начале был адрес `172.28.x.x` — это сеть самого UTM.
- **Мост (bridged)**: виртуалка выглядит для роутера как отдельное устройство и получает адрес в той же Wi-Fi-сети (`10.210.125.x`). Для лабы нужен именно он.

`ping -c 4 <IP>` — проверить, что узел отвечает

`ping` шлёт маленькие пакеты протокола **ICMP** («ты тут?») и ждёт ответа. `-c 4` — отправить 4 штуки и остановиться. В выводе смотри на два числа:

- `0% packet loss` — все пакеты дошли, связь есть;
- `time=8.59 ms` — сколько шёл пакет туда и обратно. По Wi-Fi это единицы-десятки миллисекунд; `0.04 ms` значит, что ты пингуешь сам себя.

Важное различие: успешный `ping` доказывает только то, что **хост жив**. Работает ли конкретная программа на порту 5000, он не проверяет — для этого нужен `curl`.

AP Isolation

Это настройка роутера, которая запрещает устройствам в одной Wi-Fi-сети видеть друг друга (часто включена в кафе и университетах). Раз `ping` прошёл в обе стороны, у вас она была выключена.

3. SSH: безопасный вход на чужой компьютер

SSH (Secure Shell) — способ зайти в терминал другого компьютера по сети. Всё, что идёт по каналу, зашифровано. **SCP** — копирование файлов поверх того же SSH. Именно так мы передавали `clean_data.csv` с А на Б.

Клиент и сервер

- **Клиент** (`ssh`) — программа, которая подключается. Её настройки лежат в `/etc/ssh/ssh_config`.
- **Сервер** (`sshd`, d = daemon, фоновая служба) — сидит на порту 22 и принимает подключения. Его настройки — в `/etc/ssh/sshd_config`.

Разница в одну букву `d`, и ты на ней один раз споткнулся (см. «Разбор ошибок»). Для передачи А→Б сервер обязателен на Б, а клиент нужен на А.

Команды установки и запуска

Команда	Что делает
`sudo apt install ssh`	ставит клиент и сервер OpenSSH
`sudo ssh-keygen -A`	создаёт **ключи хоста** — «паспорт» сервера, по которому клиенты его узнают (`-A` = все недостающие типы)

Команда	Что делает
`sudo systemctl enable --now ssh`	`enable` — запускать при включении компьютера, `--now` — и прямо сейчас тоже
`sudo sshd -t`	проверяет конфиг на ошибки (`-t` = test). Пустой вывод = всё хорошо
`sudo systemctl restart ssh`	перезапускает сервер, чтобы он перечитал настройки
`sudo systemctl status ssh`	показывает состояние. Ищи `active (running)` и `Server listening on 0.0.0.0 port 22`

`systemctl` — это пульт управления службами в Linux (systemd). Те же команды работают для Docker: `systemctl status docker`.

Две настройки безопасности

Мы не правили большой `sshd_config`, а положили свои строки в отдельный файл `/etc/ssh/sshd_config.d/00-lab2.conf`. Основной конфиг подтягивает всё из этой папки директивой `Include`. Так чище и легко откатить.

- `PermitRootLogin no` — нельзя зайти по SSH под `root`. Имя `root` есть на каждом Linux, поэтому боты перебирают пароли именно к нему.
- `MaxAuthTries 3` — не больше 3 попыток входа за одно подключение, потом сервер рвёт соединение. Ты это видел вживую: после неверного пароля пришло `Too many authentication failures`. Почему после двух паролей, а не трёх? Клиент сначала молча предложил серверу свой SSH-ключ, и это тоже засчиталось попыткой.

Команда, которой файл записывался:

```
printf 'PermitRootLogin no\nMaxAuthTries 3\n' | sudo tee
/etc/ssh/sshd_config.d/00-lab2.conf
```

`printf` печатает две строки (`\n` — перенос). `|` (пайп) передаёт их следующей команде. `sudo tee файл` записывает их в файл с правами root. Просто `sudo echo ... > файл` не сработал бы: перенаправление `>` выполняется с правами обычного пользователя.

Вход по ключу вместо пароля

Ключ — это пара файлов:

- `~/.ssh/lab2_ed25519` — **приватный** ключ. Это как сам ключ от замка: лежит только у тебя, никому не отдаётся.
- `~/.ssh/lab2_ed25519.pub` — **публичный** ключ. Это замок: его можно раздавать и вешать на серверы.

Порядок действий:

1. `ssh-keygen -t ed25519 -f ~/.ssh/lab2_ed25519` — создать пару. `-t` задаёт алгоритм (Ed25519 — современный и короткий), `-f` — имя файла. Парольную фразу можно оставить пустой: иначе скрипт в контейнере не сможет её ввести.
2. `ssh-copy-id -i ~/.ssh/lab2_ed25519.pub ilu-shaa@10.210.125.250` — дописать публичный ключ в файл `~/.ssh/authorized_keys` друга. Это список «замков», которые сервер принимает. Пароль друга нужен один раз.
3. `ssh -i ~/.ssh/lab2_ed25519 ilu-shaa@10.210.125.250` — теперь вход без пароля.

Как это работает: сервер присылает случайную задачку, а клиент подписывает её приватным ключом. Сервер проверяет подпись публичным. Сам приватный ключ по сети не передаётся никогда.

Отпечаток хоста и known_hosts

При первом подключении ты видел:

```
The authenticity of host '10.210.125.250' can't be established.
ED25519 key fingerprint is: SHA256:fUiiQnpiR/...
Are you sure you want to continue connecting (yes/no)?
```

Это защита от подмены сервера. Ответ `yes` записывает отпечаток в `~/.ssh/known_hosts`. Если кто-то потом подставит другой компьютер с тем же IP, ssh заметит, что отпечаток другой, и откажется подключаться. Скрипт в контейнере использует именно этот файл (`StrictHostKeyChecking=yes`).

4. Docker: контейнеры

Контейнер — это изолированный процесс со своей файловой системой, своими библиотеками и своим IP-адресом. Похож на маленькую виртуалку, но гораздо легче: ядро Linux у всех контейнеров одно, общее с хостом. Зачем это нужно: у тебя ARM, у друга x86, а Python 3.9 с pandas и scikit-learn в контейнерах ведут себя одинаково. И систему мы не засоряем.

Три понятия

Понятие	Аналогия	У нас
Dockerfile	рецепт	«возьми `python:3.9-slim`, поставь ssh и curl, поставь pandas, скопируй скрипт»
Образ (image)	готовое блюдо в морозилке, не меняется	`node_a-data-processor`, `node_b-ml-trainer`
Контейнер	блюдо на тарелке: запущенный экземпляр образа	`data-processor`, `ml-trainer`

Из одного образа можно запустить сколько угодно контейнеров. В выводе сборки строки `[2/6] RUN apt-get ...`, `[5/6] RUN pip install ...` — это шаги рецепта. А `CACHED` во второй раз значит, что Docker взял готовый шаг из кэша. Поэтому первая сборка шла минуты (`pip install` — 260 секунд), а вторая 10 секунд.

Docker Compose

Вместо длинной команды `docker run -d --name ... -p ... -v ... -e ...` все параметры записаны в файл `compose.yaml`. Запуск — одной командой:

```
docker compose up --build -d
```

- `up` — создать и запустить всё, что описано в файле;
- `--build` — сначала пересобрать образ по Dockerfile;
- `-d` — detached, в фоне: терминал сразу освобождается.

В конце вывода три галочки: образ `Built`, сеть `node_a_default` `Created`, контейнер `Started`. Compose сам создаёт для проекта отдельную сеть — отсюда тот самый интерфейс `br-3d3d...` в `ip a`.

Почему контейнер A не завершается сразу? Контейнер живёт, пока работает его главная команда. У A это `sleep infinity` («спи вечно») — то же, что `tail -f /dev/null` из задания. А работу мы запускаем внутри вручную.

Команды, которые ты запускал

Команда	Что делает
`docker ps`	список запущенных контейнеров: имя, образ, статус `Up`, порты. `--filter name=ml-trainer` — показать только один
`docker exec data-processor <команда>`	выполнить команду **внутри** уже работающего контейнера
`docker exec -it data-processor sh`	зайти в контейнер с интерактивным терминалом: `-i` — ввод с клавиатуры, `-t` — терминал. Выход — `exit`
`docker exec -i data-processor python - <<'EOF' ... EOF`	так ты запускал функцию очистки. `<<'EOF'` (heredoc) отдаёт несколько строк кода на вход Python в контейнере, а `-` значит «читай код со стандартного ввода»
`docker cp файл data-processor:/home/`	скопировать файл с хоста в контейнер. Так скрипт попал в `/home/`
`docker compose down`	остановить и удалить контейнеры и сеть проекта (образы остаются)

Проброс порта: `5000:5000`

У контейнера свой внутренний IP (`172.18.0.2`). С другого ноутбука он не виден: эта сеть существует только внутри хоста Б. Запись `ports: - "5000:5000"` читается как «**порт хоста : порт контейнера**»: всё, что придёт на `10.210.125.250:5000`, Docker перешлёт в `172.18.0.2:5000`. В `docker ps` это видно как `0.0.0.0:5000->5000/tcp`. Здесь `0.0.0.0` значит «на всех сетевых интерфейсах хоста».

Контейнеру A проброс не нужен: он сам ничего не раздаёт, а только ходит наружу. Исходящие соединения из контейнера Docker и так пропускает через NAT.

Тома (volumes): общая папка хоста и контейнера

Запись `- ./data:/home/DATA` значит: папка `node_b/data` на хосте и папка `/home/DATA` в контейнере — это одна и та же папка. Ключевой трюк лабы:

1. А кладёт файл по SCP в обычную папку хоста Б (`/mnt/Laba2/node_b/data`).
2. В контейнере `ml-trainer` этот файл сразу появляется как `/home/DATA/clean_data.csv` .

Суффикс `:ro` = read-only: контейнер может читать, но не писать. Так же в контейнер А монтируется твоя папка `~/.ssh` (в `/run/ssh`): скрипт берёт оттуда ключ и `known_hosts` , но изменить их не может.

`.env` — настройки без правки кода

Адрес Б, имя пользователя и путь на Б не зашиты в скрипт. Они лежат в `node_a/.env` , Compose передаёт их в контейнер как переменные окружения, а Python читает их через `os.getenv("NODE_B_HOST")` . Сменилась сеть — правишь одну строчку в `.env` .

Про `sudo`

У друга все docker-команды идут через `sudo` : Docker управляется службой от root. Чтобы обходиться без sudo, пользователя добавляют в группу `docker` . В задании просили другой вариант: разрешить `sudo docker` без пароля. Это делается строкой в `sudo visudo` : `ilu-shaa ALL=(root) NOPASSWD: /usr/bin/docker` .

5. iptables: сетевой экран

Итог раздела: на Б мы добавили два правила, и теперь порт 5000 отвечает только твоему ноутбуку (10.210.125.97), а всем остальным в Wi-Fi молча отказывает.

iptables — инструмент для правил, по которым ядро Linux решает, что делать с каждым сетевым пакетом: пропустить, выбросить или перенаправить.

Таблицы, цепочки, правила

- **Таблица** — тип работы. `filter` (по умолчанию) решает «пускать или нет». `nat` подменяет адреса.
- **Цепочка** — точка на пути пакета. `INPUT` — пакет для самого хоста. `FORWARD` — пакет проходит сквозь хост дальше, например в контейнер. `OUTPUT` — исходящий.
- **Правило** — «если пакет такой-то, то сделай вот это». Правила проверяются **сверху вниз**, и срабатывает первое подходящее.

Что делает сам Docker

Когда ты пишешь `-p 5000:5000`, Docker сам дописывает правила:

- в таблице `nat` (цепочка `DOCKER`) — правило **DNAT**: «пакет на порт 5000 хоста переадресовать на 172.18.0.2:5000». Именно так и работает «проброс порта»;
- в таблице `filter` — разрешение пропускать этот трафик и служебные цепочки `DOCKER-USER`, `DOCKER-ISOLATION-STAGE-1/2`.

Посмотреть эти правила можно командами `sudo iptables -t nat -nvL` и `sudo iptables -nvL`. Ключи: `-t nat` — выбрать таблицу, `-L` — показать список, `-n` — адреса цифрами без DNS-имён, `-v` — подробно, со счётчиками пакетов.

Почему именно `DOCKER-USER`

Трафик в контейнер идёт не через `INPUT`, а через `FORWARD`: пакет предназначен не самому хосту, а контейнеру за ним. Поэтому правило в `INPUT` не помогло бы. А свои цепочки `DOCKER` Docker перезаписывает при каждом запуске. `DOCKER-USER` — цепочка, которую Docker оставил специально для пользователя. Она проверяется первой и никогда не трогается.

Два наших правила

```
sudo iptables -I DOCKER-USER 1 -s 10.210.125.97 -p tcp --dport 5000 -j ACCEPT
sudo iptables -I DOCKER-USER 2 -p tcp --dport 5000 -j DROP
```

Кусок	Значение
`-I DOCKER-USER 1`	вставить (Insert) в цепочку на позицию 1
`-s 10.210.125.97`	если источник (source) — твой ноутбук
`-p tcp --dport 5000`	и это TCP на порт назначения 5000
`-j ACCEPT` / `-j DROP`	то прыгнуть (jump) в действие: пропустить / молча выбросить

Порядок решает всё. Пакет от тебя совпадёт с правилом 1 и проходит. Пакет от кого-то ещё не совпадает с первым, доходит до правила 2 и выбрасывается. Если поменять правила местами, DROP сработает и для тебя.

Проверка — `sudo iptables -nvL DOCKER-USER --line-numbers`. Должно быть `1 ACCEPT ... 10.210.125.97 ... tcp dpt:5000` и `2 DROP ... 0.0.0.0/0 ... tcp dpt:5000`; `0.0.0.0/0` здесь значит «любой адрес».

Разница DROP и REJECT: DROP молча выбрасывает пакет, и чужой `curl` просто висит до таймаута. REJECT сразу отвечает «отказано».

Сохранение правил

Правила iptables живут в памяти и пропадают после перезагрузки. Поэтому друг сделал так:

1. `sudo apt install -y iptables-persistent` — пакет, который при старте системы загружает сохранённые правила. Он заодно удалил `ufw`: это другая надстройка над iptables, и вместе они конфликтуют.
2. `sudo netfilter-persistent save` — записать текущие правила в `/etc/iptables/rules.v4` (и `rules.v6` для IPv6).

Альтернатива из задания — `sudo iptables-save > файл`. Она просто выводит все правила текстом, а `iptables-restore < файл` их возвращает.

6. Скрипты: что делает код

Два скрипта на Python: клиент `process_and_request.py` на А и сервер `ml_service.py` на Б. Весь код с комментариями есть в отчёте, здесь — суть.

Данные: набор Iris

150 цветков ириса трёх видов (`setosa`, `versicolor`, `virginica`), по 50 каждого. У каждого четыре измерения в сантиметрах: длина и ширина чашелистика (sepal) и лепестка (petal). Задача модели — по четырём числам угадать вид. Это классический учебный набор, он встроен в scikit-learn. Выданного CSV у вас не было, поэтому ты сгенерировал `raw_data.csv` из него, и получилось `(150, 5)`: 150 строк, 5 столбцов.

Узел А: `process_and_request.py`

1. **Загрузка.** `pd.read_csv(...)` читает CSV в таблицу pandas (DataFrame).
2. **Очистка** — функция `clean_data`:
   - `dropna(how="all")` удаляет полностью пустые строки, потом `dropna()` — строки, где пропущено хотя бы одно значение;
   - `.str.strip().str.lower()` у текстовых столбцов убирает пробелы по краям и переводит в нижний регистр, чтобы `" Setosa"` и `"setosa"` стали одним классом;
   - проверки: данные не пусты, есть столбец `species` и в нём хотя бы 2 класса.

У вас вышло `150 0 (150, 5)`: удалить нечего, в Iris нет пропусков. Это нормально.

3. **Передача** — `transfer_csv`. Из Python через `subprocess.run` запускаются обычные команды `ssh ... mkdir -p` (создать папку на Б) и `scp clean_data.csv ilu-shaa@10.210.125.250:/mnt/Laba2/node_b/data/`. Опция `BatchMode=yes` означает «никогда не спрашивай пароль»: если ключ не подошёл, сразу ошибка.
4. **Запрос прогноза** — `request_predictions`: `curl http://<IP_B>:5000/predict` с повторами, если сервер ещё не готов. Потом `json.loads` разбирает ответ.

Узел Б: `ml_service.py`

Обучающая и тестовая выборки. Проверять модель на тех же данных, на которых она училась, — как дать на экзамене те же задачи, что были в домашке. Поэтому `train_test_split` делит 150 строк на 120 для обучения (80 %) и 30 для проверки (20 %).

- `stratify=labels` — в обеих частях виды в одинаковой пропорции: в тесте 10 + 10 + 10;
- `random_state=42` — фиксированная «случайность», чтобы при каждом запуске выходил один и тот же результат. Поэтому у тебя и у друга одинаковые `row_index` и accuracy.

Дерево решений (DecisionTreeClassifier) — это игра в «да/нет». Модель сама подбирает вопросы, которые лучше всего разделяют классы. Например: «длина лепестка < 2,45 см?» — да → setosa; нет → следующий вопрос про ширину лепестка, и так далее. `fit` строит дерево на 120 строках, `predict` прогоняет через него 30 тестовых.

accuracy = 0,9333 — доля верных ответов: 28 из 30. Ошибки обычно между versicolor и virginica — эти виды похожи, а setosa отличается сильно.

`Pipeline` + `ColumnTransformer` + `OneHotEncoder` — задел на случай текстовых признаков (например, столбец «цвет»): их превращают в числа 0/1, потому что дерево в scikit-learn понимает только числа. В Iris все признаки числовые, так что тут это не срабатывает.

Flask и REST API

Flask — минималистичный веб-сервер на Python. Декоратор `@app.get("/predict")` значит: когда придёт GET-запрос на `/predict`, вызвать эту функцию. А `jsonify(...)` превращает словарь Python в JSON-ответ.

Эндпоинт	Что отвечает
`GET /health`	«я жив» + есть ли файл данных. Нужен для быстрой проверки, ничего не обучает
`GET /predict`	`status`, `train_rows: 120`, `test_rows: 30`, `accuracy`, список `predictions`

Расшифровка одного элемента ответа: `{"prediction":"setosa","row_index":38}` — «строку № 38 очищенного CSV модель считает цветком setosa» (счёт с нуля).

Коды ошибок: `200` — всё хорошо (виден в журнале Flask), `503` — данных ещё нет, `422` — данные кривые.

`app.run(host="0.0.0.0", port=5000)` — важная строчка. С `host="127.0.0.1"` сервер слушал бы только сам контейнер изнутри, и проброшенный порт не работал бы.

На фото друга ещё есть `train_model.py` и поля `model_ready` / `test_data_ready` в `/health`. Это его вариант того же сервиса: обучение вынесено в отдельный шаг, а модель сохраняется в файл `model.joblib`. Смысл тот же.

Предупреждение **`This is a development server`** в логе Flask — не ошибка. Встроенный сервер подходит для учёбы, а в продакшене ставят gunicorn и подобные.

7. Путь одного запроса

Всё, что было выше, сходится в одной команде `docker exec data-processor curl http://10.210.125.250:5000/predict`. Вот как пакет проходит через сеть, Docker и iptables.



путь запроса curl от контейнера А до контейнера Б и обратно

Если любая остановка сломана, запрос не дойдёт. Поэтому отлаживают по порядку: сначала `ping` до хоста, потом `curl /health` с самого Б (`127.0.0.1`), потом с А. Контейнеру А проброс порта не нужен: он только ходит наружу, а ответ возвращается по уже открытому соединению. В журнале Flask запрос виден от адреса `172.18.0.1` — это шлюз docker-моста на Б, через него входят все запросы снаружи.

8. Разбор ошибок, которые у вас случились

На сдаче понимание ошибок ценится не меньше, чем успешный результат. Вот все, что видны на скринах.

Сообщение	Что случилось	Как починили
`ipconfig: command not found`	`ipconfig` — команда Windows (а `ipconfig getifaddr en0` — macOS). В Linux её нет	`ip a`
`Unknown command verb 'ssh'`	написал `systemctl ssh` без действия	`systemctl status ssh`

Сообщение	Что случилось	Как починили
`Bad configuration option: permitrootlogin`	серверные директивы записаны в конфиг **клиента** `/etc/ssh/ssh_config.d/`. Клиент `ssh` таких слов не знает и вообще перестал запускаться	удалили файл и записали в `/etc/ssh/sshd_config.d/`
`Permission denied` → `Too many authentication failures`	неверный пароль (и в команде `ilu-sha` вместо `ilu-shaa`). Сработал `MaxAuthTries 3`	это не баг, а доказательство, что настройка работает
`SyntaxError: invalid syntax` в `python -c "..."`	несколько команд Python в одной строке без `;` между ними	расставили `;`
`cannot import name 'clead_data'`	опечатка в имени функции	`clean_data`
`subprocess.CalledProcessError ... ssh ... exit status 255`	код 255 — у ssh это «не удалось подключиться/авторизоваться». Скрипт полагается только на ключ (`BatchMode=yes`), а ключ ты пересоздал (`Overwrite? y`) и новую публичную часть на Б не скопировал	снова `ssh-copy-id` нового ключа. Файл к тому моменту уже лежал на Б
`host/network 'IP_A' not found`	в команду iptables вставили заглушку из README вместо настоящего IP	`-s 10.210.125.97`
`Index of insertion too big`	первое правило не добавилось, цепочка пустая, а вставлять на позицию 2 в пустой цепочке нельзя	сначала ACCEPT на 1, потом DROP на 2
`Address already in use / Port 5000 is in use`	в контейнере уже работал другой экземпляр Flask. Один порт — одна программа	остановить старый процесс или просто использовать уже запущенный
`^[[200~curl ...: command not found`	служебные символы вставки из буфера (bracketed paste) попали в команду	набрали команду заново
`WARN ... Bake, but buildx isn't installed`	предупреждение, не ошибка: нет плагина для нового сборщика, и Compose собрал образ старым способом	ничего

Общий урок: в сетевых проблемах иди слоями. Порядок такой: `ping` (хост доступен?) → `ssh` руками (авторизация работает?) → `curl /health` (сервис отвечает?) → `curl /predict` (логика работает?).

9. Шпаргалка команд

Все команды лабы в порядке выполнения. В столбце «Где»: А/Б — на хосте, «конт. А» — внутри контейнера.

Шаг	Команда	Где	Зачем
1	`ip a` / `ip -br -4 addr`	А, Б	узнать свой IP
1	`ping -c 4 10.210.125.250`	А	видит ли А ноутбук Б
1	`sudo systemctl status docker`	А, Б	Docker запущен?
2	`sudo apt install ssh`	А, Б	поставить OpenSSH
2	`printf '...' | sudo tee /etc/ssh/sshd_config.d/00-lab2.conf`	А, Б	PermitRootLogin no, MaxAuthTries 3
2	`sudo sshd -t`	А, Б	проверить конфиг
2	`sudo systemctl restart ssh` + `status ssh`	А, Б	применить и проверить
3	`docker compose up --build -d`	А, Б	собрать образ и запустить контейнер
3	`docker ps`	А, Б	контейнер `Up`, порты
3	`docker exec -it data-processor sh`	А	зайти внутрь контейнера
3	`docker exec data-processor ping -c 4 10.210.125.250`	конт. А	видит ли контейнер А ноутбук Б
4	`sudo iptables -t nat -nvL`, `sudo iptables -nvL`	Б	посмотреть правила Docker
4	`sudo iptables -I DOCKER-USER 1 -s 10.210.125.97 -p tcp --dport 5000 -j ACCEPT`	Б	пустить А
4	`sudo iptables -I DOCKER-USER 2 -p tcp --dport 5000 -j DROP`	Б	всех остальных не пускать

Шаг	Команда	Где	Зачем
4	`sudo netfilter-persistent save`	Б	сохранить правила навсегда
5	`docker exec data-processor python -c "...load_iris..."`	конт. А	создать `raw_data.csv`
5	`docker cp process_and_request.py data-processor:/home/`	А	скопировать скрипт в контейнер
5	`docker exec data-processor ls -l /home /home/DATA`	конт. А	проверить, что файлы на месте
6	`ssh-keygen -t ed25519 -f ~/.ssh/lab2_ed25519`	А	создать пару ключей
6	`ssh-copy-id -i ~/.ssh/lab2_ed25519.pub ilu-shaa@10.210.125.250`	А	отдать публичный ключ на Б
6	`ssh -i ~/.ssh/lab2_ed25519 ilu-shaa@10.210.125.250`	А	проверить вход без пароля
6	`docker exec data-processor python /app/process_and_request.py`	конт. А	весь пайплайн: очистка → scp → curl
7	`sudo docker exec -it ml-trainer python /app/ml_service.py`	Б	запустить Flask
7	`curl http://127.0.0.1:5000/health`	Б	проверить сервис локально
7	`docker exec data-processor curl http://10.210.125.250:5000/predict`	конт. А	**главная проверка лабы**

Если преподаватель попросит повторить запрос со своего ноутбука (это есть в задании), помни: правило DROP его заблокирует. На Б нужно добавить ещё один ACCEPT для его IP **выше** DROP:

```
sudo iptables -I DOCKER-USER 1 -s <IP_преподавателя> -p tcp --dport 5000 -j ACCEPT
```

Или покажи наоборот: его `curl` виснет, а твой проходит. Это наглядно доказывает, что файрвол работает.

10. Вопросы для самопроверки перед сдачей

Попробуй ответить вслух, потом сверься с коротким ответом.

1. **Чем контейнер отличается от виртуальной машины?** — У виртуалки своё ядро и эмулируемое «железо». Контейнер — просто изолированный процесс на ядре хоста, поэтому стартует за секунды и весит мало.
2. **Что означает `-p 5000:5000` и зачем он нужен?** — Порт хоста : порт контейнера. Без него контейнер с адресом 172.18.0.2 недоступен с другого компьютера. Docker реализует это правилом DNAT в iptables.
3. **Почему фильтруем в `DOCKER-USER`, а не в `INPUT`?** — Трафик к контейнеру идёт через `FORWARD`, а `DOCKER-USER` — единственная цепочка, которую Docker не перезаписывает.
4. **Что будет, если поменять ACCEPT и DROP местами?** — Правила читаются сверху вниз. DROP окажется первым и заблокирует всех, включая А.
5. **Зачем `netfilter-persistent save`?** — Правила iptables хранятся в памяти и исчезают после перезагрузки. Команда пишет их в `/etc/iptables/rules.v4`.
6. **В чём разница `ssh_config` и `sshd_config`?** — Первый — настройки клиента (кто подключается), второй — сервера (к кому подключаются). `PermitRootLogin` и `MaxAuthTries` — серверные.
7. **Зачем запрещать вход под root?** — Имя `root` есть везде, и у него полные права: это главная цель перебора паролей. Правильнее заходить обычным пользователем и повышать права через `sudo`.
8. **Чем ключ лучше пароля?** — Ключ не передаётся по сети, его невозможно подобрать перебором, и скрипты могут работать без ввода пароля.
9. **Что такое `known_hosts`?** — Список отпечатков серверов, которым ты доверяешь. Защищает от подмены сервера (атака «человек посередине»).
10. **Как файл попадает с А внутрь контейнера на Б?** — SCP кладёт его в папку хоста Б, а эта папка смонтирована в контейнер как том (`./data:/home/DATA`).
11. **Почему Flask слушает `0.0.0.0`, а не `127.0.0.1`?** — `127.0.0.1` — только изнутри контейнера. Пакеты через проброс приходят на его сетевой интерфейс (172.18.0.2), и их нужно принимать.
12. **Зачем делить данные на train и test?** — Чтобы честно оценить модель на данных, которых она не видела. Иначе дерево просто запомнит ответы, и accuracy будет 1,0.
13. **Что значит accuracy 0,9333?** — 28 из 30 тестовых цветков классифицированы верно.
14. **Что проверяет `ping`, а что — `curl`?** — `ping` (ICMP) проверяет, что хост доступен. `curl` (TCP/HTTP) проверяет, что конкретный сервис на порту 5000 отвечает.
15. **Что такое AP Isolation?** — Настройка роутера, которая запрещает Wi-Fi-клиентам общаться напрямую. С ней лаба невозможна.